LAB ASSIGNMENT-13.4

SINDHU KATAKAM

2303A52481

BATCH-41

Task 1: Refactoring Data Transformation Logic

PROMPT:

Refactor the code using list comprehension . include docstring inline commets and proper explanation of the code.

Legacy code:

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Refactored code:

```
"""
Refactored using list comprehension

This script takes a list of integer values and creates a new list containing
each value doubled. The refactoring uses list comprehension for concise and
efficient code.
"""

values = [2, 4, 6, 8, 10]  # Original list of integers

# List comprehension to double each value in 'values'
doubled = [v * 2 for v in values]  # Inline comment: multiplies each element by 2

print(doubled)  # Output: [4, 8, 12, 16, 20]

# Explanation:
# - List comprehension provides a compact way to process each item in a list.
# - '[v * 2 for v in values]' means: for each 'v' in 'values', multiply 'v' by 2 and collect the results in a new list.
```

Output:

```
PS D:\AI_ASSIT_CODING> &
0.3.py
[4, 8, 12, 16, 20]
```

Justification:

This code starts with a list of numbers called values. It uses a list comprehension to make a new list called doubled, where each number from the original list is multiplied by two. The list comprehension makes the code shorter and easier to read. After creating the new list, the code prints it out. This is a simple way to quickly process and change all items in a list

Task 2: Improving Text Processing Code Readability

PROMPT:

Refactor the code while keeping the final output unchanged. refactor code as efficient and readable approach . include docstring,inline comments and explanation of the code.

Legacy code:

```
words = ["Refactoring", "with", "AI", "improves",
"quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Refactored code:

```
"""
Assignment 10.3 - Refactored for efficiency and readability

This script takes a list of words and combines them into a single sentence,
separated by spaces. The refactored version uses the efficient 'join' method
to concatenate the words.
"""

words = ["Refactoring", "with", "AI", "improves", "quality"]  # List of words

# Use 'join' to efficiently concatenate words with a space separator
message = " ".join(words)  # Inline comment: joins all words with spaces

print(message)  # Output: Refactoring with AI improves quality

# Explanation:
# - The 'join' method is more efficient than concatenating strings in a loop.
# - It takes all items in the list and joins them into a single string, separated by the specified delimiter (a space in this case).
# - This approach improves both readability and performance.
```

Output:

```
PS D:\AI_ASSIT_CODING> & C:\Users\MYSELF'
Refactoring with AI improves quality
```

Justification:

The legacy code uses a loop to add each word to a sentence one by one, which makes the code longer and less clear. It also adds extra spaces that need to be removed at the end. The refactored code uses the join method, which combines all the words in one step and is easier to read. This method is also faster and uses less memory. Overall, the refactored code is simpler, cleaner, and more efficient than the legacy code

Task 3: Safer Access to Configuration Data

Prompt:

review the legacy code that manually checlks for dictionary keys. refactor the code using safer dictionary access methods.make it more efficient and readable approach . ensure the behaviour remains the same for missing keys . include docstring, inline comments, and explanation

Legacy code:

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Refactored code:

```
"""
Assignment 10.3 - Refactored for safer dictionary access

This script checks for the presence of a 'timeout' key in a configuration dictionary.
The refactored version uses the safer and more concise 'get' method to access dictionary values,
providing a default message if the key is missing.
"""

config = {"host": "localhost", "port": 8080}  # Configuration dictionary

# Use the 'get' method to safely access 'timeout', with a default message if not found
timeout = config.get("timeout", "Default timeout used")  # Inline comment: returns value or default

print(timeout)  # Output: Default timeout used

# Explanation:
# - The 'get' method allows safe access to dictionary keys without raising a KeyError if the key is missing.
# - If 'timeout' exists, its value is returned; otherwise, the default message is used.
# - This approach makes the code shorter, clearer, and less error-prone.
```

Output:

```
PS D:\AI_ASSIT_CODING> & (
Default timeout used
```

Justification:

The legacy code checks if the key exists in the dictionary before trying to use it, which makes the code longer and harder to read. It also repeats the key name, which can lead to mistakes. The refactored code uses the get method, which does both checking and retrieving in one step. This makes the code shorter, clearer, and less likely to cause errors. Overall, the refactored code is simpler and safer to use.

Task 4: Refactoring Conditional Logic for Scalability

Prompt:

review the legacy code and examine the multiple if-elif conditions used to determine operations. refactor the logic using mapping technique while preserving functionality such that code looks cleaner ,scalable alternative. .include docstring ,inline comments ,and explanation

Legacy code:

```python
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Refactored code:

```python
"""
Assignment 10.3 - Refactored using mapping for cleaner and scalable operation selection

This script performs a mathematical operation (add, subtract, multiply, divide) on two numbers
based on the value of the 'action' variable. The refactored version uses a dictionary mapping
to associate actions with their corresponding functions, making the code cleaner and easier to extend.
"""

action = "divide"
x, y = 10, 2

# Mapping actions to corresponding lambda functions
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}

# Use the mapping to get the correct operation, or return None if action is invalid
result = operations.get(action, lambda a, b: None)(x, y)  # Inline comment: safely calls the operation

print(result)  # Output: 5.0

# Explanation:
# - The 'operations' dictionary maps each action string to a function that performs the operation.
# - The 'get' method retrieves the correct function or a default function returning None if the action is not found.
# - This approach is cleaner, more readable, and makes it easy to add new operations in the future.
```

Output:

```
PS D:\AI_ASSIT_CODING>
5.0
```

Justification:

The legacy code uses many if-elif statements to check which operation to do, making it longer and harder to update. Each new operation would need another if or elif line, which can get messy. The refactored code uses a dictionary to map actions to their functions, so it is much shorter and easier to read. Adding new operations is simple—just add another item to the dictionary. This makes the code cleaner, easier to manage, and better for future changes.

Task 5: Simplifying Search Logic in Collections

Prompt:

review the legacy code . identify the explicit loop used for searching an item.make it more efficient and readable approach .nclude docstring,inline comments,and explanation

Legacy code:

```python
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Refactored code:

```python
"""
Refactored for efficient item search in a list

This script checks if the item "eraser" is present in the inventory list.
The refactored version uses the 'in' keyword for a more efficient and readable search.
"""

inventory = ["pen", "notebook", "eraser", "marker"]  # List of items in inventory

# Use 'in' keyword for efficient and readable membership test
found = "eraser" in inventory  # Inline comment: returns True if "eraser" is in the list

print("Item Available" if found else "Item Not Available")  # Output: Item Available

# Explanation:
# - The 'in' keyword checks for the presence of an item in a list directly, making the code concise.
# - This approach is more efficient than looping through each item manually.
# - It improves readability and reduces the chance of errors.
```

Output:

```
PS D:\AI_ASSIT_CODING>
Item Available
```

Justification:

The legacy code uses a loop to check each item in the list one by one, which makes the code longer and harder to read. It also needs extra steps to set and check a flag. The refactored code uses the in keyword, which checks for the item in a single, simple line. This makes the code much shorter and easier to understand. Overall, the refactored code is clearer, faster, and less likely to have mistakes.